

Day 7 — Design Component (25 Aug 2026)

1. What the Design Tab Is For

Custom iFlows, Interfaces — this is the tab where you build your own integration content (as established on Day 6).

2. Package Structure Recap



About Separate Folders: different types of documents can be organized within a package — not just iFlows, but supporting documentation too, as we'll see below.

3. BTP Cockpit vs. CPI Main Page

BTP Cockpit → this leads you to the **CPI main page** — the entry point into your actual Integration Suite tenant, where you then see: **Discover, Design, and Monitor** (the tabs covered across Days 5–7).

Now our work is in the Design Tab.

4. Creating a Package

Package (Create) → kind of a folder.

Example used in class: S4/HANA → Commerce Cloud (the same scenario from Day 6, now being used as the actual package you're creating).

Important rule: While naming, Spaces are not allowed. Package names must be continuous, without blank spaces.

Fields Required When Creating a Package

1. **Name**
2. **Technical Name**
3. **Short Description**

5. Tabs Within a Package

Header, Overview, Artifact, Documents, Comments, Tags

Header Tab

Header → **Which integration** (i.e., identifies which specific integration this package represents). Fields:

Field	Notes
Name	
Technical Name	Special characters are not allowed
Short Description	
Version	
Vendor	
Mode	Editable

Overview Tab

Summary / Description → **overview of the flow.** This is where you write a plain-language explanation of what the package/flow actually does.

Artifacts Tab

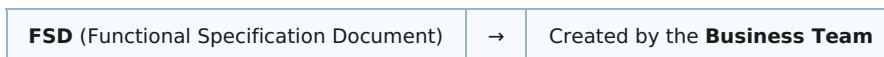
Create, Place objects → Add (dropdown). This is where you actually create new iFlows and other objects inside the package, using an "Add" dropdown menu.

Documents Tab

You can attach different supporting document types to a package:

Document Type	Full Form / Purpose
Technical Documents	Technical-level details of the integration
Functional Documents	Business-process-level details
Business Documents	General business context/documentation
FSD	Functional Specification Document

FSD and TSD — Who Creates What



Correction: Your notes have "**TSD → Technical Specification System.**" The correct expansion is **TSD = Technical Specification Document** (not "System"). The TSD is the technical counterpart to the FSD — while the FSD describes *what* the business needs (written by the business team), the TSD describes *how* it will be technically built (written by the technical/development team, i.e., you as the CPI developer), and the actual **iFlow** is built based on the TSD.



6. What We Need to Know on Both Sides (Sender & Target)

For both the Sender System and the Target System, we need to know:

- **Sender System, Target System**
- **Adapters**
- **Authentication method**
- **Format we need to send**, and **format we are getting from the Source system**

7. When Is Transformation Required?

Any transformation required — this comes up whenever the source and target systems use different formats or structures.

Worked Example — Honda to MRF via Salesforce (Revisited)



We will transfer the data from Source to Target (Sender to Receiver).

8. Adapters

Definition: Adapters are used to establish the connection between one system and another system.

Multiple Adapters — Examples by System

System	Common Adapters Used
S4/HANA	IDoc, RFC, Proxy, OData, HTTP, SOAP
Salesforce	HTTP, OData, SOAP
File	FTP, SFTP
Database	JDBC

Worked Example — Format Conversion Across the Flow

S4/HANA (IDoc, XML format)	→	CPI	HTTP →	Third Party (JSON format, OAuth etc.)
--------------------------------------	---	------------	--------	---

9. Authentication Methods

#	Method	How It Works
1	Basic	Username + Password
2	OAuth	Token-based
3	Client Certificate	Certificate-based authentication
4	JWT	JSON Web Token
5	Bearer	Token-based; grant type = Client Credentials

10. Data Formats

Data formats: XML, JSON, CSV, Text.

Important correction: Your notes say "**CPI will only support XML Format.**" This is not accurate and is worth correcting clearly, since it could cause confusion in an interview. **SAP CPI actually supports a wide range of formats** — including XML, JSON, CSV, EDI, and plain text — through its various adapters and converters, not XML alone.

What's likely true, and probably what your trainer was getting at: **CPI's graphical Message Mapping step has historically worked natively with XML internally** — meaning when you map a JSON payload, CPI often converts it to XML first (or uses a dedicated JSON-to-XML / XML-to-JSON converter step) before/after the graphical mapping step. So while the *mapping engine's internal working format* leans on XML, the platform as a whole (via adapters and converter steps like XML↔JSON, XML↔CSV) fully supports JSON, CSV, and other formats in real production scenarios. If your trainer meant this narrower, more technical point, it's worth asking them to clarify in class so your notes reflect the precise distinction.

11. Transformation — Mapping Example

Example from class:

Source (Text/XML format): `Satish` → Target (JSON format): `Kumar`

The point being illustrated: **Satish → Kumar** represents a **Mapping / Transformation** — i.e., a value or field from the source is transformed/converted into a corresponding value or field in the target, potentially changing both the value and the overall format (Text/XML → JSON) at the same time.